



Advertisement: Support JavaWorld, click here!

November 2000

HOME

FEATURED
TUTORIALS

COLUMNS

NEWS &
REVIEWS

FORUM

JW
RESOURCES

ABOUT
JW

Validation with Java and XML Schema, Part 3

Parse XML Schema to validate data

Summary

Taking validation beyond simple if-then-else structures, XML Schemas can provide a better way to validate data in Java applications. In this article, Brett continues to build the validation framework and fill in the blanks left from the last article. You'll learn to parse the XML Schema, build up Java representations of the schema's constraints, and apply those constraints to an application's data. *(3,600 words)*

By Brett McLaughlin

You've seen it happen. Heck, you've probably been a part of the problem more than a few times yourself. The problem? Validation. We, as programmers, pride ourselves on lugging our toolbox of code, tips, tricks, and experience to the jobs and projects on which we work. But in every application, when it comes to validation, we seem to lean towards reinventing the wheel. We write and rewrite code, and end up missing out on a great chance to add to our toolboxes.

Read the whole "Validation with Java and XML Schema" series:

- [Part 1. Learn the value of data validation and why pure Java isn't the complete solution for handling it](#)
- [Part 2. Use XML Schema for constraining Java data](#)
- [Part 3. Parsing XML Schema to validate data](#)
- [Part 4. Build Java representations of schema constraints and apply them to Java data](#)

On the other hand, everyone (and their dog!) is looking to get into XML. Using the Extensible Markup Language seems to be even more popular than hacking the Linux kernel these days and will even make your boss happy. So how do those two fit together? Well, XML, and specifically XML Schema, provides a perfect means of detailing constraints for Java data. And with a simple Java-based framework, you can build those constraints into Java objects and compare application data against them. The end result is a flexible, robust, XML-based framework for all your Java validation needs.

In this article, I'll show you how to parse an XML Schema and build up a set of constraints. Once those constraints are ready for your Java program to use, I'll detail the process of comparing data to them. Finally, your application will be given a means to pass in data and determine which

constraint to apply to that data, indicating if that data is valid for that constraint. But before diving in, let me fill you in on what's already happened in this series.

If you missed the premiere

In [Part 1](#), I spent a lot of time talking about validation in general terms. I looked at some common bad practices you tend to find in validation code, particularly the case of simply hard coding in constraints. Of course, that is not at all portable, so I also looked at some utility classes, such as Jason Hunter's `ParameterParser` class from *Java Servlet Programming*. That class allows the simple conversion from the `String` format in which servlets receive data to various other Java formats such as `ints`, `floats`, and `booleans`. However, that still did not address other common validation needs such as range checking and specifying only a few allowed values. Finally, I introduced XML as a possible solution, showing how an XML document is superior to Java's standard property files.

In [Part 2](#), I introduced the validation framework. Starting with some basic design, I showed the four basic classes you need to code:

- The `Constraint` class, which represents constraints for a single type, like the `shoeSize` type.
- The `Validator` class, which provides an interface for allowing developers to pass in data, and find out if the data is valid.
- The `schemaParser` class, which parses an XML Schema and creates the `Constraint` objects for use by the `Validator` class.
- The `DataConverter` helper class, which will convert from XML Schema data types to Java data types, and perform other data type conversions for you.

In that article, I showed you the `Constraint` class in its entirety, providing basic methods that allowed setting an allowed range, a data type, and allowed values for the data. If you were to add additional constraint types, such as pattern matching, you would add them to that class. I also outlined the `Validator` class, and left a blank where schema parsing would occur, which I will fill in this article.

So now you are ready to dive into the guts, right? In this article, I'll start with parsing the XML Schema, and building up constraints. Next, you'll see how to take those constraints and apply them to data in the `Validator` class. So let's get to it.

Parsing the schema

The bulk of the work that you need to do is in the `schemaParser` class. That class has one single task: to parse an XML Schema. While it parses, it should take each XML Schema attribute and create a `Constraint` instance out of it. To refresh your memory, here's a sample XML Schema. That is still based on the shoe store that was discussed in the previous articles but has some additional constraints:

```
<?xml version="1.0"?>

<schema targetNamespace="http://www.buyShoes.com"
        xmlns="http://www.w3.org/1999/XMLSchema"
        xmlns:buyShoes="http://www.buyShoes.com"
>

    <attribute name="shoeSize">
```

```

    <simpleType baseType="integer">
      <minExclusive value="0" />
      <maxInclusive value="20" />
    </simpleType>
  </attribute>

  <attribute name="width">
    <simpleType baseType="string">
      <enumeration value="A" />
      <enumeration value="B" />
      <enumeration value="C" />
      <enumeration value="D" />
      <enumeration value="DD" />
    </simpleType>
  </attribute>

  <attribute name="brand">
    <simpleType baseType="string">
      <enumeration value="Nike" />
      <enumeration value="Adidas" />
      <enumeration value="Dr. Marten" />
      <enumeration value="V-Form" />
      <enumeration value="Mission" />
    </simpleType>
  </attribute>>

  <attribute name="numEyelets">
    <simpleType baseType="integer">
      <minInclusive value="0" />
    </simpleType>
  </attribute>

</schema>

```

As an example, the `schemaParser` would parse the attribute for the constraint named "shoeSize" and create a new instance of the `Constraint` class. That class would have a data type of "int." Notice that it doesn't have "integer" because that is an XML Schema data type. Instead, the `Constraint` class converts that data type (using the `DataConverter` class) to the Java equivalent. It will then have a value of 0 for the minimum (exclusive) value, and a value of 20 for the maximum (inclusive) value. In that constraint, the minimum (inclusive), maximum (exclusive), and allowed values will all not be used, as they aren't specified; another constraint might specify allowed values but no range at all. So with that in mind, I'll start with looking at the class skeleton.

The `schemaParser` skeleton

The `schemaParser` class has few public methods. The class's constructor takes in a `java.net.URL` pointing to the XML Schema to parse. That method will then need to fire off a private method, which will handle the parsing and build up constraints. Once the instance has been constructed and parsing has occurred, the client needs to access the built-up constraints. To allow that, two methods are provided: `getConstraints()`, which returns a list of the `Constraint` objects resulting for a parse, and `getConstraint(String constraintName)`, which returns the `Constraint` for the supplied name (if it exists).

Here, then, is the skeleton for this class. It takes care of importing all the various classes that will be needed, and defines the storage that will be used by the `parseschema()` method. Once that

skeleton is in place, I'll show you how to handle the actual parsing.

```
package org.enhydra.validation;

import java.io.IOException;
import java.net.URL;
import java.util.Map;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;

// JDOM classes used for document representation
import org.jdom.Attribute;
import org.jdom.Document;
import org.jdom.Element;
import org.jdom.JDOMException;
import org.jdom.Namespace;
import org.jdom.input.SAXBuilder;

import org.enhydra.validation.Constraint;

/**
 * <p>
 * The <code>schemaParser</code> class parses an XML Schema and creates
 * <code>{@link Constraint}</code> objects from it.
 * </p>
 */
public class schemaParser {

    /** The URL of the schema to parse */
    private URL schemaURL;

    /** The constraints from the schema */
    private Map constraints;

    /** XML Schema Namespace */
    private Namespace schemaNamespace;

    /** XML Schema Namespace URI */
    private static final String SCHEMA_NAMESPACE_URI =
        "http://www.w3.org/1999/XMLSchema";

    /**
     * <p>
     * This will create a new <code>schemaParser</code>, given
     * the URL of the schema to parse.
     * </p>
     *
     * @param schemaURL the <code>URL</code> of the schema to parse.
     * @throws <code>IOException</code> - when parsing errors occur.
     */
    public schemaParser(URL schemaURL) throws IOException {
        this.schemaURL = schemaURL;
    }
}
```

```

        constraints = new HashMap();
        schemaNamespace =
            Namespace.getNamespace(SCHEMA_NAMESPACE_URI);

        // Parse the schema and prepare constraints
        parseschema();
    }

    /**
     * <p>
     * This will return constraints found within the document.
     * </p>
     *
     * @return <code>Map</code> - the schema-defined constraints.
     */
    public Map getConstraints() {
        return constraints;
    }

    /**
     * <p>
     * This will get the <code>Constraint</code> object for
     * a specific constraint name. If none is found, this
     * will return <code>null</code>.
     * </p>
     *
     * @param constraintName name of constraint to look up.
     * @return <code>Constraint</code> - constraints for
     *         supplied name.
     */
    public Constraint getConstraint(String constraintName) {
        Object o = constraints.get(constraintName);
        if (o != null) {
            return (Constraint)o;
        } else {
            return null;
        }
    }

    /**
     * <p>
     * This will do the work of parsing the schema.
     * </p>
     *
     * @throws <code>IOException</code> - when parsing errors occur.
     */
    private void parseschema() throws IOException {
        // Parse the schema and build up constraints
    }
}

```

Pretty straightforward so far, right? Good. You should notice that the constructor and `parseschema()` method can both throw an `IOException` if problems arise. That gives the code a

means of reporting problems back up the chain to the client, using the validation framework. It also is the type of Exception that any Java code using the URL class might generate, which means that the code doesn't have to trap for those sorts of errors; instead, they are just thrown up the calling chain.

Another important item you should note is the `schemaNamespace` variable. That variable holds the JDOM Namespace object for the XML Schema namespace. If you look at the XML Schema document again (shown above), the XML Schema namespace URI is assigned to the default namespace. That means that all nonprefixed elements in the schema (which happens to be all elements) are assigned to that default namespace, the XML Schema namespace. Getting the associated JDOM Namespace object for that default, XML Schema, namespace will help you look up items in the document; in the next section, I'll show you how that all fits together.

Once you have the skeleton in place, you need to handle actually parsing the XML Schema. I'll discuss that now.

XML, schemas, and JDOM

The key to the entire `schemaParser` class is being able to (no surprise here) actually parse an XML Schema. Many XML parsers, such as Apache Xerces, currently offer options for schema validation; however, you do not want to use those facilities. In fact, you don't want the XML Schema to be handled as a schema at all. That is because all parsers, at least in their current versions, use vendor-specific structures for handling XML Schemas. The result is nonportable code, the enemy of any Java programmer.

Instead, the `schemaParser` class can rely on the fact that an XML Schema document is actually an XML document as well. It conforms to XML's well-formedness rules and, therefore, can be treated as any other XML document. Therefore, the schema parser can read in the XML Schema as an XML document and operate on it as it would any other document with which it works. That is exactly what the `parseschema()` method does.

Using JDOM, which you can obtain in [Resources](#), the `parseschema()` method first uses SAX to read in the supplied schema URL and build a JDOM Document object. And now is when that `schemaNamespace` variable comes into play (remember I said it would?). The XML Schema `attribute` construct represents all the constraints, so once the document is read into memory, those constraints are located within the document by simply looking up all elements named `attribute` (I know, it's sort of confusing, isn't it? All the `attribute` elements...) in the XML Schema namespace. Then, each of the resulting objects (represented by a JDOM Element) are passed to a utility method, `handleAttribute()`. The code shown here puts that into action:

```
/**
 * <p>
 * This will do the work of parsing the schema.
 * </p>
 *
 * @throws <code>IOException</code> - when parsing errors occur.
 */
private void parseschema() throws IOException {
    /**
     * Create builder to generate JDOM representation of XML Schema,
     * without validation and using Apache Xerces.
     */
    SAXBuilder builder = new SAXBuilder();
```

```

    try {
        Document schemaDoc = builder.build(schemaURL);

        // Handle attributes
        List attributes = schemaDoc.getRootElement()
                               .getChildren("attribute",
                                             schemaNamespace);

        for (Iterator i = attributes.iterator(); i.hasNext(); ) {
            // Iterate and handle
            Element attribute = (Element)i.next();
            handleAttribute(attribute);
        }

        // Handle attributes nested within complex types

    } catch (JDOMException e) {
        throw new IOException(e.getMessage());
    }
}

```

That is fairly straightforward and matches the concepts that I just walked through. The `getChildren()` method returns a Java `List` of elements matching the criteria supplied; the code then iterates through that `List`, peeling off each element, representing a constraint, and invokes the `handleAttribute()` method. Now I'll show you that method, which does the real work.

Handling attributes

I'm not going to spend a lot of time walking through the `handleAttribute()` method; by now, you're starting to understand how the `SchemaParser` class works and probably starting to see how simple JDOM makes things as well. The `handleAttribute()` method receives a JDOM `Element`, which represents a data constraint. Its task is to create a new constraint and add it to the constraint list in the class instance.

First, a new `Constraint` is created. `handleAttribute()` then begins to run through all of the various options that a constraint can have and retrieves the data for each option. If data is present, it sets the data for the `Constraint` instance. If not, it simply moves on to the next constraint option. In the method shown here, the name, data type, allowed values, and ranges on a piece of data are examined and set:

```

/**
 * <p>
 * This will convert an attribute into constraints.
 * </p>
 *
 * @throws <code>IOException</code> - when parsing errors occur.
 */
private void handleAttribute(Element attribute)
    throws IOException {

    // Get the attribute name and create a Constraint
    String name = attribute.getAttributeValue("name");
    if (name == null) {

```

```

        throw new IOException("All schema attributes must have names.");
    }
    Constraint constraint = new Constraint(name);

    // See if there is a data type on this constraint
    String schemaType = attribute.getAttributeValue("type");
    if (schemaType != null) {
        constraint.setDataType(
            DataConverter.getInstance().getJavaType(schemaType));
    }

    // Get the simpleType - if none, we are done with this attribute
    Element simpleType = attribute.getChild("simpleType",
schemaNamespace);
    if (simpleType == null) {
        return;
    }

    // Handle the data type
    schemaType = simpleType.getAttributeValue("baseType");
    if (schemaType == null) {
        throw new IOException("No data type specified for constraint " +
name);
    }
    constraint.setDataType(DataConverter.getInstance().getJavaType
(schemaType));

    // Handle any allowed values
    List allowedValues = simpleType.getChildren("enumeration",
schemaNamespace);
    if (allowedValues != null) {
        for (Iterator i=allowedValues.iterator(); i.hasNext(); ) {
            Element allowedValue = (Element)i.next();
            constraint.addAllowedValue(allowedValue.getAttributeValue
("value"));
        }
    }

    // Handle ranges
    Element boundary = simpleType.getChild("minExclusive",
schemaNamespace);
    if (boundary != null) {
        Double value = new Double(boundary.getAttributeValue("value"));
        constraint.setMinExclusive(value.doubleValue());
    }
    boundary = simpleType.getChild("minInclusive", schemaNamespace);
    if (boundary != null) {
        Double value = new Double(boundary.getAttributeValue("value"));
        constraint.setMinInclusive(value.doubleValue());
    }
    boundary = simpleType.getChild("maxExclusive", schemaNamespace);
    if (boundary != null) {
        Double value = new Double(boundary.getAttributeValue("value"));
        constraint.setMaxExclusive(value.doubleValue());
    }

```



```

    }
    boundary = simpleType.getChild("maxInclusive", schemaNamespace);
    if (boundary != null) {
        Double value = new Double(boundary.getAttributeValue("value"));
        constraint.setMaxInclusive(value.doubleValue());
    }

    // Store this constraint
    constraints.put(name, constraint);
}

```

Nothing here is too magical. If you have a need for other constraints, such as pattern matching or, perhaps, more complex data types, you can add enhancements to the `handleAttribute()` method. For items such as pattern matching, you would want to add additional code to obtain the pattern element within the constraint and work with that value:

```

Element pattern = simpleType.getChild("pattern", schemaNamespace);
if (pattern != null) {
    String patternValue = pattern.getAttributeValue("value");
    // Set this pattern on the Constraint object
}

```

You would also need to make a change to the `Constraint` class and add a couple more methods. Making a change to the data types, though, involves a change to the helper class you see used here, `DataConverter`. That class handles conversion from an XML Schema type to a Java type such as from `integer` (schema type) to `int` (Java type). You could add new data types to `DataConverter`, perhaps as XML Schema matures or as your needs increase. `DataConverter` is included, of course, in the [source code](#) available for download with this article.

Once this method has completed, the `schemaParser` constructor will complete, and return control to the invoking program. At that point, the invoking program (the original `Validator` class constructor) can use the `getConstraints()` or `getConstraint()` method to obtain the data constraints and work with them. In fact, that's exactly what the `Validator` method, `isValid()`, does! So I'll return to that now.

Data validation

So now I'm going to move back to the front line, the `Validator` class. If you remember, that is the actual class with which your clients will work, and the `schemaParser` class stays behind the scenes. The `isValid()` method, specifically, is what a client would use to check a specific piece of data against its constraints. That method would be used like this:

```

String shoeSize = req.getParameterValue("shoeSize");
if (!Validator.getInstance().isValid("shoeSize", shoeSize)) {
    // Report an error back to the client
}

// Continue using the shoe size

```

That is as simple as it gets -- you would supply a data value (in `String` format) and then the name of the constraint to validate against. `isValid()` will then return whether or not the data is

valid against that constraint. You can use that result to either report an error back to an application client or continue on, knowing that the data is valid for the constraints you specified in your XML Schema.

The `isValid()` method is actually not very complex, as all of the work to obtain the data constraints was done in the `schemaParser` class. First, the method obtains the `Constraint` object for the supplied constraint name; if no constraint exists for that name, the value `true` is returned. Basically, no constraint is equivalent (in this implementation) to any data being allowed. You could make that same method throw an `Exception` or take other error actions if you desired.

Once the `Constraint` object is obtained from the `schemaParser` class, validation actually starts. The code below shows the outline of that method, and has comment placeholders for each area of validation. I'll look at each of those in turn, next.

```

/**
 * <p>
 * This will validate a data value (in <code>String</code> format)
against a
 * specific constraint, and return <code>true</code> if that value is
valid
 * for the constraint.
 * </p>
 *
 * @param constraintName the identifier in the constraints to validate
this data against.
 * @param data <code>String</code> data to validate.
 * @return <code>boolean</code> - whether the data is valid or not.
 */
public boolean isValid(String constraintName, String data) {
    // Validate against the correct constraint
    Object o = constraints.get(constraintName);

    // If no constraint, then everything is valid
    if (o == null) {
        System.out.println("No constraint found for " + constraintName);
        return true;
    }
    Constraint constraint = (Constraint)o;

    // Check data type

    // Check allowed values

    // Check ranges

    // If we got here, all tests were passed
    return true;
}

```

Again, nothing is surprising here, right? It's worth saying at this point that I'm also trying to illustrate some basic design principles to you here. You are all probably saying, "Gee, I can't believe how simple this is. Is this guy really getting paid to do this?" Well, actually I am! (<grin/>) Seriously, this particular series is a lot of basic code, and I've walked you through it in the exact

way that I coded it. But when you throw all the code together into a package, it looks a lot more complex. Many of you have asked me via email, "Can you send me the code? I've read Part 2, and it's just not enough to get me going." I have a feeling though, that you could all code this package if you took it piece by piece, in the way that I'm walking through it. What is the moral of the story? Start with a design (Parts 1 and 2) and begin to code in the client interface (Part 2). Then fill in the blanks, one at a time (Part 3, this article). Finally, test the classes and add any enhancements you need (Part 4, due next month). At the end of that process, you'll be pleasantly surprised at how much better your software is.

OK, I'm climbing off my soapbox now! In any case, let's go on to the specific validation areas involved in the `isValid()` method.

Data types

First, the data type is checked. You do that simply by using a utility method, `correctDataType`. I'm not going to show that method because it would take a lot of space for very little meat. It takes the data type supplied by the `getDataType()` method in the `Constraint` class, and converts the `String` data into that type. If conversion fails, the method obviously returns `false`. Otherwise, the data is valid (at least for that data type), and the method returns `true`. Here is the code that you need to use for data type validation:

```
// Validate data type
if (!correctDataType(data, constraint.getDataType())) {
    return false;
}
```

Allowed values

Checking for allowed values is even easier. The `Constraint` instance is checked to see if there is a list of allowed values for the data constraint. If there is, then the values are obtained, as a `Java List`, through the `getAllowedValues()` method. Then, the supplied data is checked to see if it occurs in the returned list of values. If so, then validation can continue; however, if the value is not found, the method halts and returns `false`. Here's the code for that functionality:

```
// Validate against allowed values
if (constraint.hasAllowedValues()) {
    List allowedValues = constraint.getAllowedValues();
    if (!allowedValues.contains(data)) {
        return false;
    }
}
```

One note here: you might want to be careful with the `getAllowedValues()` method. Keep in mind that the checking that occurs in that case will be *case-sensitive*. If you want the comparison to be *case-insensitive*, resulting in `v-form` being treated as equal to `V-Form`, you would need to iterate through the list yourself. For each value, you could use the `equalsIgnoreCase()` method on `java.lang.String` and see if the values are equal without regard to case. That is a modification you might consider in your own applications.

Range checking

The last piece of validation to perform is checking the value range. That is a bit trickier because XML Schema provides two types of minimum values (inclusive and exclusive) and two types of maximum values (again, inclusive and exclusive). While you might be tempted to take the inclusive

values and subtract (for minimum) or add (for maximum) a very small value and only use a floor and ceiling value, you would lose precision in that area. It's simply too risky, as those values might need to be very precise. Instead, the code uses different operands for each. Those are shown here:

Boundary Operator

Minimum (Exclusive) <=

Minimum (Inclusive) <

Maximum (Exclusive) >=

Maximum (Inclusive) >

Of course, all these numeric functions must be performed against a numeric value. So, the code first converts the String value into a Java double. The following code will perform that range checking:

```
// Validate against range specifications
try {
    double doubleValue = new Double(data).doubleValue();
    if (constraint.hasMinExclusive()) {
        if (doubleValue <= constraint.getMinExclusive()) {
            return false;
        }
    }
    if (constraint.hasMinInclusive()) {
        if (doubleValue < constraint.getMinInclusive()) {
            return false;
        }
    }
    if (constraint.hasMaxExclusive()) {
        if (doubleValue >= constraint.getMaxExclusive()) {
            return false;
        }
    }
    if (constraint.hasMaxInclusive()) {
        if (doubleValue > constraint.getMaxInclusive()) {
            return false;
        }
    }
} catch (NumberFormatException e) {
    // If it couldn't be converted to a number, the data type isn't
    // numeric anyway, as it would have already failed,
    // so this can be ignored.
}
```

Believe it or not, that's all there is to it! The `isValid()` method is now complete, which in turn means that the `Validator` class is complete. Your clients are now ready to import a few classes and then use the validation framework shown here. And while it might seem that that is the end of the road, that isn't the case. First, I want to mention a few changes made in the code (based partly on the great feedback I received) that were not shown here. And then, I have a report on the extension of this series, which means that I'll be back on this topic one month from now.

Evolution of an API

Like any good piece of code, the validation classes are constantly changing. Since I wrote the last article, I've received a ton of feedback, which is terrific. Among that feedback were a few helpful emails from people working with my code and catching some subtle errors. A method was not synchronized when it should have been, some code was comparing a numeric value to the numeric constant `Double.NaN` (a test that will *never* return true), and a few other things. That is great, as it improves the code, and that's what open source is all about! In any case, I've made those corrections, as well as some that I've found on my own, and updated the source code that you can download from the [Resources](#) section of this article. If you have code from the last article, you should get the updates there. And, if all goes well, you and I will uncover even more tidbits for the next article, and the code will be updated there as well!

Summary

I hope you've enjoyed filling in many of the holes left out of the framework skeleton in the last article. The code now is usable and could be put into place in many applications. However, some improvements still need to be made. When validation errors occur, a simple false value is returned without indication of what the problem was. That is obviously lacking, as any good application should not only validate data but also inform the client on what errors were encountered in processing the data. Additionally, I still haven't shown you an example of this code in action. For those reasons, I'm happy to report that *JavaWorld* has agreed to extend this series by one more part. In the next, and final (really, I promise!) article, I'll show you how to modify the code to return more information about errors that occur. A simple Exception hierarchy will be examined for the code, and I'll discuss how you can use that hierarchy to better report errors. Finally, I'll show you a realistic example, using servlets, for using the validation code in an application. I hope you've enjoyed what we've covered so far and that you'll come back next month for the final piece. See you online until then. 

About the author

[Brett McLaughlin](#) is an Enhydra strategist at Lutris Technologies who specializes in distributed systems architecture. He is the author of [Java and XML](#) and is involved in technologies such as Java servlets, Enterprise JavaBeans technology, XML, and business-to-business applications. With Jason Hunter, he recently founded the [JDOM](#) project, which provides a simple API for manipulating XML from Java applications. McLaughlin is also an active developer on the Apache Cocoon project and the EJBoss EJB server, and a cofounder of the Apache Turbine project.

Resources

- Download the source code for this article:
<http://www.javaworld.com/jw-11-2000/Validation/src.zip>
 - Catch the beginning of it all in Part 1 of this series:
<http://www.javaworld.com/javaworld/jw-09-2000/jw-0908-validation.html>
 - Follow up by reading Part 2:
<http://www.javaworld.com/javaworld/jw-10-2000/jw-1013-validation2.html>
- Curious about JDOM? Get into it with these *JavaWorld* articles on the subject:
- "Easy Java/XML Integration with JDOM, Part 1," Jason Hunter and Brett McLaughlin (*JavaWorld*, May 2000): <http://www.javaworld.com/javaworld/jw-05-2000/jw-0518-jdom.html>
 - "Easy Java/XML Integration with JDOM, Part 2," Jason Hunter and Brett McLaughlin (*JavaWorld*, July 2000): <http://www.javaworld.com/javaworld/jw-07-2000/jw-0728-jdom2.html>
- Get JDOM today from the Website:
<http://www.jdom.org/>
 - Want to know more about XML Schema? Visit the XML Schema homepage:
<http://www.w3.org/XML/schema.html>
 - Find out more about the Enhydra application server:
<http://www.enhydra.org/>

- "Objects, Objects Everywhere," Brett McLaughlin (IBM DeveloperWorks, July 2000) a data-binding series:
<http://www-4.ibm.com/software/developer/library/data-binding1/index.html?dwzone=xml>
- *Java and XML*, Brett McLaughlin (O'Reilly, 2000) -- pick Brett's brain on all things XML by reading his book:
<http://www.oreilly.com/catalog/javaxml>
- Discuss Java and XML in *JavaWorld's* Java and XML forum, moderated by Mark Johnson:
<http://forums.idg.net/webx?50@@.ee6b78f>
- For more XML-related articles, check out the *JavaWorld* Topical Index:
<http://www.javaworld.com/javaworld/topicalindex/jw-ti-javaxml.html>



Advertisement: Support JavaWorld, click here!

[HOME](#) | [FEATURED TUTORIALS](#) | [COLUMNS](#) | [NEWS & REVIEWS](#) | [FORUM](#) | [JW RESOURCES](#) | [ABOUT JW](#) | [FEEDBACK](#)

Copyright © 2002 JavaWorld.com, an IDG Communications company